Méthodologie RAD

- Livrer vite ce qui compte · Journée RAD Quest



À la fin de la journée, vous saurez...

Décrire les 4 phases du RAD

Prioriser un backlog avec MoSCoW

Travailler en timebox : réduire le périmètre, pas allonger le temps

Livrer un prototype web par itérations, avec le feedback du client

Comparer RAD, cascade et Agile, et citer leurs héritiers actuels

Repérer les pièges sécurité et données personnelles d'un prototype

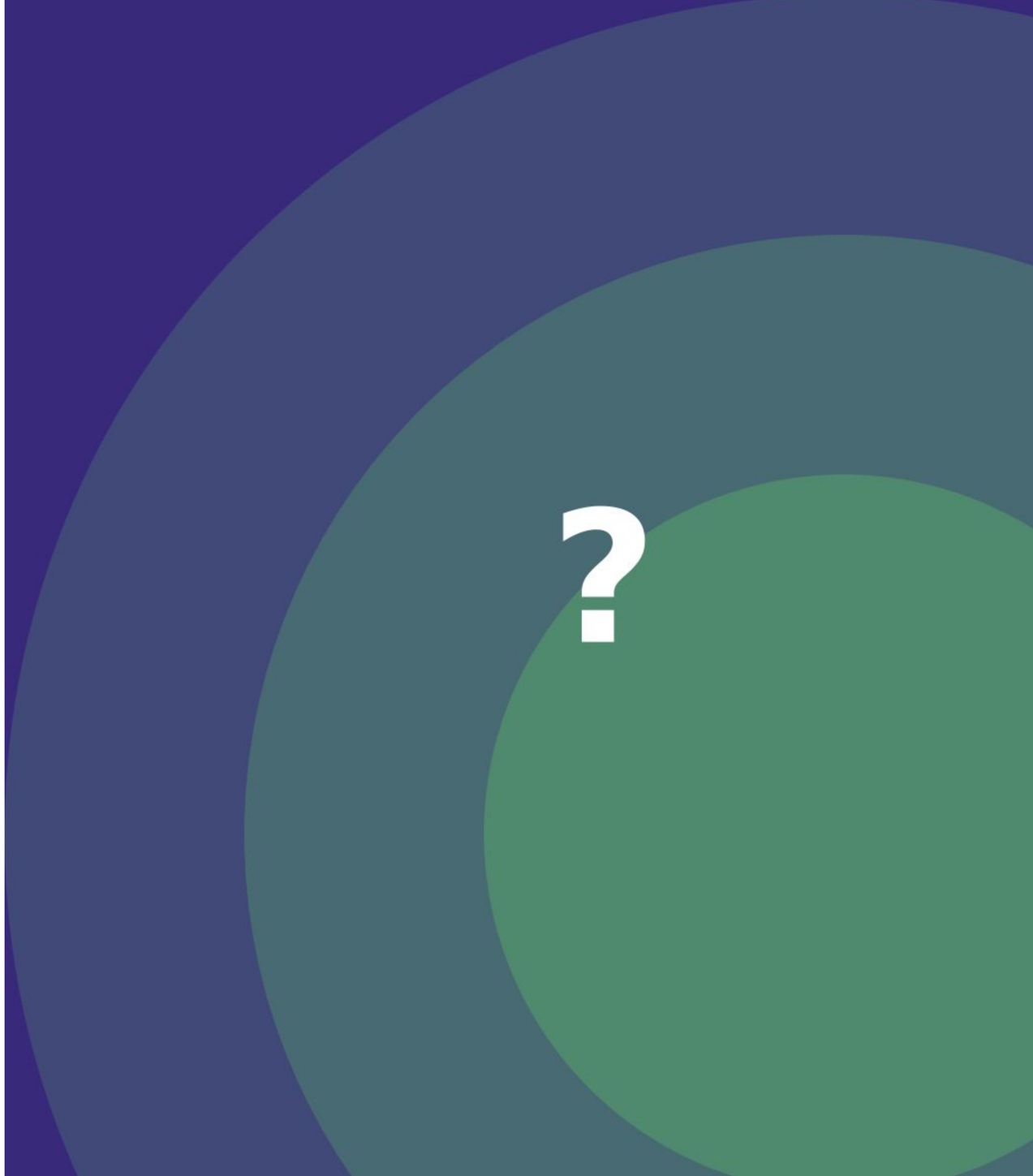
Programme : 45 minutes de théorie, puis 7 heures de jeu

1. D'où vient le RAD ?
2. Les 4 phases
3. Les outils : MoSCoW, timebox, prototype
4. Le RAD en 2026
5. Les règles de RAD Quest

Ensuite : planification → 3 cycles de conception → 2 sprints de construction → cutover → debrief

D'où vient le RAD ?

1 · Histoire et définition



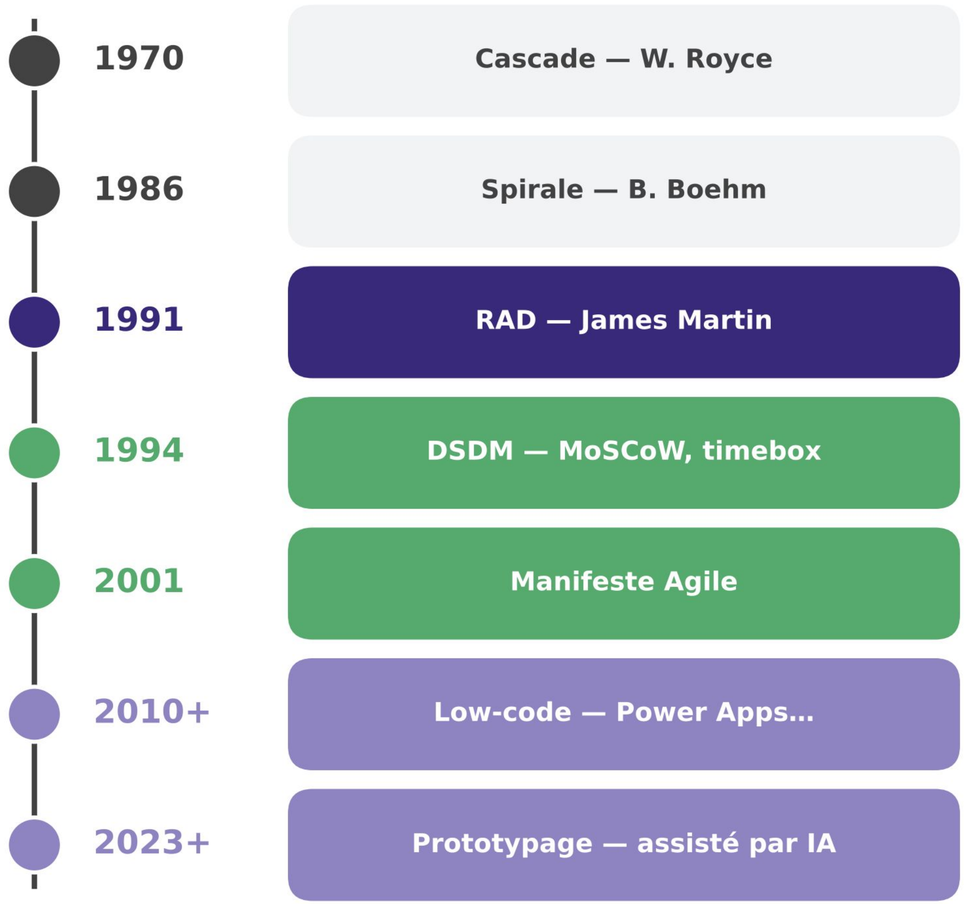
Le RAD est né contre la lenteur du cycle en cascade

Années 1970–80 : projets de 2 à 3 ans, spécifications figées au départ

Résultat fréquent : le logiciel livré ne correspond plus au besoin

1991 : James Martin publie « Rapid Application Development »

Idée : impliquer l'utilisateur et livrer en quelques mois grâce aux prototypes



RAD : livrer vite un système utile, en construisant avec l'utilisateur

Rapid Application Development : méthode de développement qui remplace les longues spécifications par des **prototypes** montrés très tôt aux utilisateurs.

Le délai est fixé à l'avance (souvent moins de 90 jours). Ce qui varie, c'est le périmètre.

Quatre principes font tenir le RAD

Prototyper plutôt que décrire : on montre, on ne rédige pas 80 pages

Impliquer l'utilisateur à chaque étape, en ateliers

Timeboxer : chaque étape a une durée fixe et non négociable

Réutiliser : composants existants, outils de génération, équipes réduites et expertes

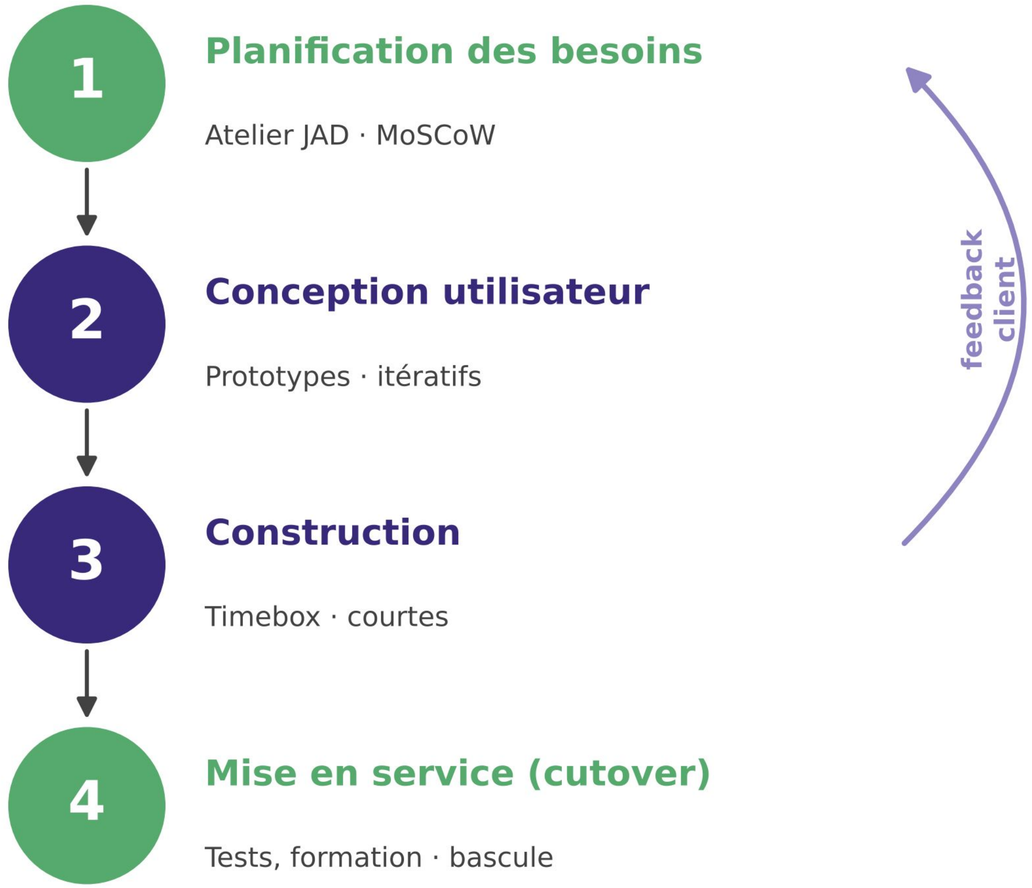
Les 4 phases

2 · Du besoin à la mise en service

1·2·3·4

Le RAD enchaîne 4 phases, avec une boucle de feedback client

1. **Planification des besoins** : on s'accorde sur le périmètre
2. **Conception utilisateur** : prototypes successifs avec le client
3. **Construction** : on finalise dans des timebox courtes
4. **Mise en service** : tests finaux, formation, bascule



Phase 1 : un atelier JAD remplace le cahier des charges

JAD (Joint Application Design) : atelier où client, utilisateurs et développeurs définissent ensemble le besoin

Objectif : une liste d'exigences priorisées, pas une spec exhaustive

Bonnes questions : « Qui l'utilise ? Quand ? Qu'est-ce qui est indispensable le jour J ? »

À éviter : proposer une solution technique avant d'avoir compris le besoin

Dans le jeu : 5 min d'interview du client, puis classement MoSCoW

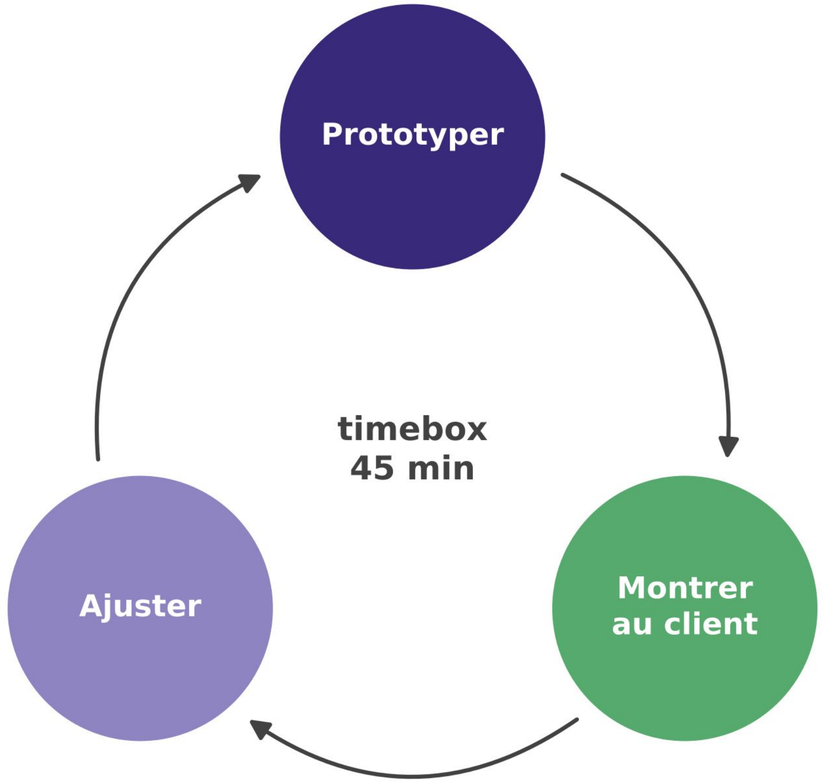
Phase 2 : on construit l'interface en 3 boucles courtes

Chaque boucle : prototyper, montrer, ajuster

Le client voit du concret dès la première heure

Un malentendu coûte 45 minutes, pas 6 mois

Après la 3e boucle, l'interface est validée et « gelée »



3 cycles, puis l'interface est « gelée »

Phases 3 et 4 : finaliser puis livrer sans surprise

Construction : terminer les Must, tester, corriger ; le client valide toujours

Les changements arrivent encore : on re-priorise au lieu de tout empiler

Mise en service (cutover) : checklist, tests, documentation, formation des utilisateurs

Dans le jeu : 8 points de checklist, de « les données survivent » à « un README existe »

Les outils du RAD

3 · MoSCoW, timebox, prototype



MoSCoW sépare l'indispensable du « ce serait bien »

- 4 niveaux : Must, Should, Could, Won't (this time)
- Popularisé par la méthode DSDM
- Règle : les Must ne dépassent pas 60 % de l'effort
- La marge de 40 % absorbe les imprévus
- « Won't » n'est pas « jamais » : c'est « pas cette fois »



La timebox fixe le temps : c'est le périmètre qui s'ajuste

Cascade : périmètre fixe, donc le délai et le coût dérapent

RAD : délai et coût fixes, donc on livre moins mais à l'heure

Pas fini ? On abandonne un Could, on ne prolonge pas

Dans le jeu : jetons d'effort non reportables

Le triangle inversé

Cascade	RAD / DSDM
FIXE **périmètre**	FIXE **temps, coût**
VARIABLE **temps, coût**	VARIABLE **périmètre**

En RAD, on réduit le périmètre, pas le délai.

Un prototype rapide ne dispense pas des bonnes pratiques

Sécurité : afficher une saisie avec textContent, jamais innerHTML (risque XSS, OWASP Top 10)

Données personnelles (RGPD) : ne collecter que le nécessaire (minimisation) ; prévoir l'effacement

Accessibilité : labels, contraste ≥ 4,5:1 (WCAG 2.2) ; l'European Accessibility Act s'applique depuis juin 2025

Dette technique : un prototype qui part en production sans revue devient un risque

Versionner : un commit par cycle

Le RAD en 2026

4 · Héritiers et comparaison

2026

Cascade, RAD et Agile répondent différemment au changement

	Cascade	RAD	Agile / Scrum
Besoins	Spécifiés en entier au départ	Ateliers + prototypes	Backlog qui évolue
Client	Au début et à la fin	À chaque démo	Product Owner dans l'équipe
Temps	Planning long	Timebox, délai court	Sprints réguliers
Risque	Produit inadapté	Dette technique	Dérive du périmètre

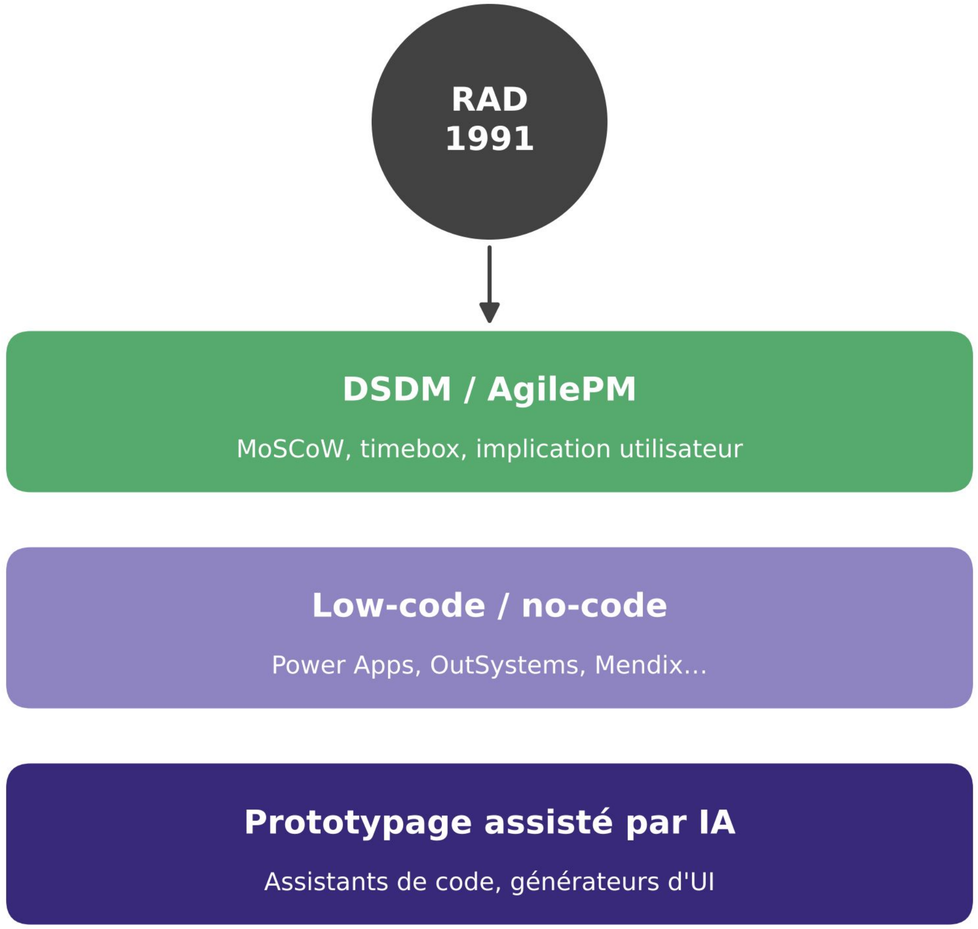
Le RAD vit aujourd'hui sous d'autres noms

DSDM / AgilePM : a gardé MoSCoW et la timebox

Low-code : Power Apps, OutSystems, Mendix ; prototyper sans tout coder

IA générative : assistants de code et générateurs d'interfaces accélèrent le prototypage

Avec l'IA : relire le code généré, et respecter l'AI Act (ex. signaler un chatbot)



RAD Quest : votre agence a une journée pour livrer

Client : le Repair Café « La Clé à Molette »

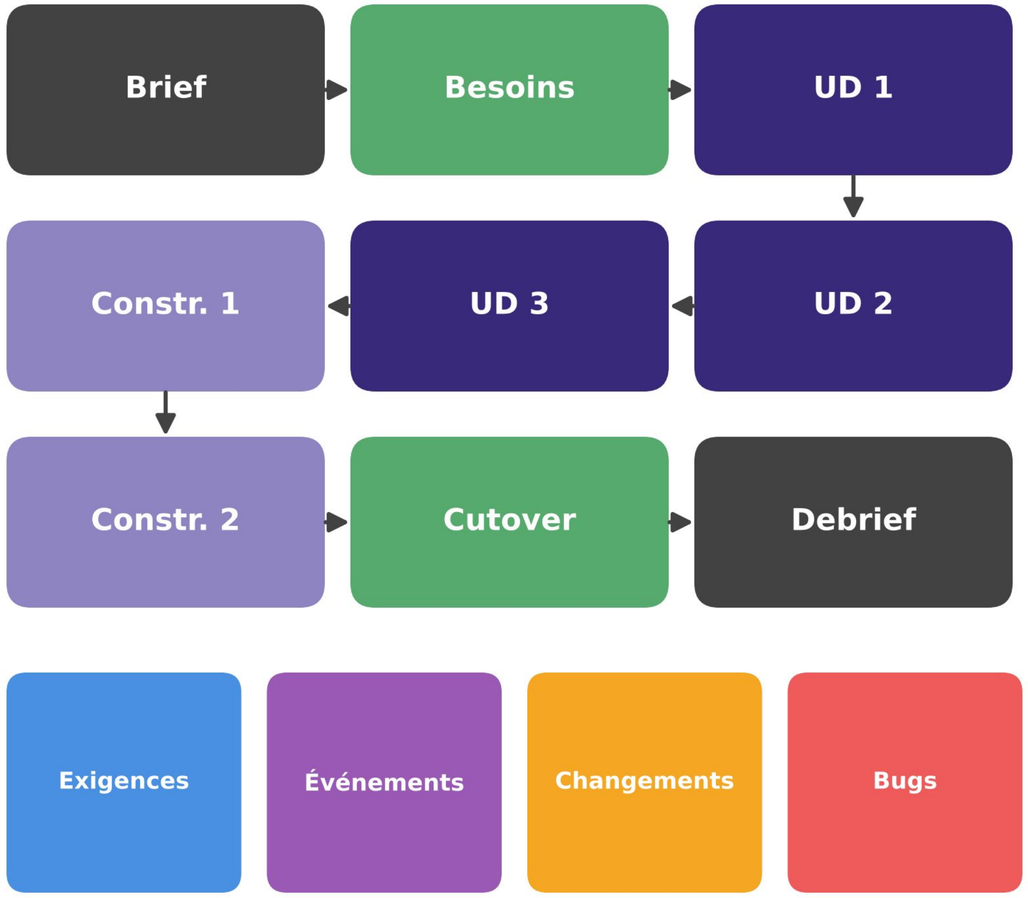
Équipes de 3–4, 9 cases sur le plateau

Jetons d'effort : 6 par cycle de conception, 8 par sprint

Une exigence compte si elle **fonctionne en démo**

4 pioches : Exigences, Événements, Changements, Bugs

Score : valeur livrée + satisfaction × 2 + bonus + cutover



À retenir

Le RAD remplace les longues spécifications par des prototypes validés par le client

4 phases : besoins, conception utilisateur, construction, mise en service

MoSCoW pour prioriser, timebox pour tenir le délai

On réduit le périmètre, jamais le délai

Vite ne veut pas dire bâclé : sécurité, RGPD et accessibilité dès le prototype

Quiz éclair : levez la main

1. Quelle phase vient après la conception utilisateur ?
2. Que signifie le W de MoSCoW ?
3. La timebox est finie, la fonctionnalité non : que faites-vous ?
4. Pourquoi éviter innerHTML avec une saisie utilisateur ?
5. Quelle méthode actuelle a repris MoSCoW ?

Sources et pour aller plus loin

James Martin, Rapid Application Development, Macmillan, 1991

Agile Business Consortium, DSDM / Agile Project Framework (MoSCoW, timeboxing)

OWASP Top 10 · W3C WCAG 2.2

RGPD (UE) 2016/679 · AI Act (UE) 2024/1689 · European Accessibility Act (UE) 2019/882

Consulté le 09/10/2026